

A Semi-Autonomous SIEM–SOAR Framework for Anomaly-Based Security Operations with Human-in-the-Loop Learning

G. Gangadara Venkateswararao
Dept. of Advanced Computer
Science

Vignan's Foundation for Science,
Technology and Research
Guntur, India
venky22vip@gmail.com
Saugat Chaudary

Dept. of Advanced Computer
Science

Vignan's Foundation for Science,
Technology and Research
Guntur, India
saugatkumarchy@gmail.com

G. S. N. S. Niteesh Babu
Dept. of Advanced Computer
Science

Vignan's Foundation for Science,
Technology and Research
Guntur, India
niteeshnani334@gmail.com
Dr. Benson Mansingh P.M

Dept. of Advanced Computer
Science

Vignan's Foundation for Science,
Technology and Research
Guntur, India
benyuva@gmail.com

V. Tarun Kumar Reddy
Dept. of Advanced Computer
Science

Vignan's Foundation for Science,
Technology and Research
Guntur, India

tharunkumarreddy4455@gmail.com

Abstract—Security Operations Centers must continuously monitor large volumes of heterogeneous security events produced by enterprise networks, cloud services, and user activities. Conventional rule-based monitoring systems are effective for known threats, but they frequently generate excessive alerts and often fail to identify stealthy or previously unseen attacks. At the same time, fully automated response mechanisms may introduce operational risk when defensive actions are executed without analyst supervision.

This paper presents a semi-autonomous SIEM–SOAR architecture that combines anomaly-based detection with guided automation and human-in-the-loop validation. CICIDS-style network flow records and synthetically generated SSH-style authentication events are collected and preprocessed using temporal aggregation and behavioral feature extraction. An Isolation Forest model is trained on baseline activity to identify deviations from normal behavior. Detected anomalies are indexed in an ELK Stack to provide centralized visibility and investigation support, while selected high-severity alerts are forwarded to a Shuffle SOAR module through webhook-based workflows that perform non-destructive notification and case-tracking actions only. Analyst feedback is modeled through a simulated feedback channel and incorporated into periodic offline retraining.

Evaluation using CICIDS-style traffic demonstrates effective identification of abnormal behavior with a manageable false-positive rate and reduced alert overload. The framework improves operational visibility and detection adaptability while preserving analyst control, illustrating a practical prototype for trustworthy SOC automation.

Index Terms—SIEM, SOAR, Security Operations Center, Anomaly Detection, Isolation Forest, Human-in-the-Loop, ELK Stack, SOC Automation

I. INTRODUCTION

Enterprise environments today produce gigabytes of operational and security data from network devices, cloud infrastructure, authentication systems, applications, and more. These logs have to be monitored continuously so that any abnormal activity and cyber threats can be detected in the early stage. Real-time monitoring is critical to ensure timely incident identification. The logs retrieved from these devices and servers are analyzed by means of SIEM in Security Operations Center. Traditional SIEM solutions parse the ingested logs based on predefined rules and name-based correlation techniques. However, traditional rule-based security solutions are not effective in detecting intricate and sophisticated attack schemes such as insider attacks, low and slow attacks, and zero-day attacks. As a result, these systems often generate many alerts, most of which are benign activity. This creates significant challenges for analysts who must sift through a high volume of false positive alerts in order to identify legitimate security incidents in a timely manner.

To address these issues, organizations increasingly adopt SOAR platforms that motivate dull effort incident handling responsibility. While automation improves response time and operational efficiency, fully autonomous security actions introduce operational risks. Automated containment actions, such as blocking hosts or disabling user accounts, may disrupt legitimate services when triggered incorrectly. Furthermore, purely automated systems lack contextual reasoning and explainability, which are essential for high-confidence security decisions in enterprise environments.

Anomaly detection based on machine learning has appeared to be a promising method for detecting abnormal behavior that signals unknown cyber attacks. Unlike rule-based approaches, these techniques learn patterns of normal behavior and flag observations that differ significantly from those patterns. As a result, abnormal patterns can be detected in both network traffic and user activity. Most literature reviews all these approaches in isolation, and the deployment of them is hardly integrated into operational SOC monitoring environments. In addition, several automated security tools do not incorporate feedback from analysts when evaluating whether an alert represents legitimate activity. This limitation reduces the dexterity of the system to acquire continuously and suit to the changes in network environments.

This paper proposes a semi-autonomous SIEM-SOAR framework that integrates unsupervised anomaly detection with controlled automation and human-in-the-loop validation. Security events are derived from CICIDS-style network flow records and synthetically generated authentication logs, then analyzed using an Isolation Forest model trained on baseline behavior. Detected anomalies are enriched with simulated contextual metadata and indexed into an ELK Stack for investigation and visualization. A Shuffle platform executes pre-defined webhook-based, non-destructive response-assistance workflows. Any potentially impactful action remains subject to analyst approval. To perform this analysis with minimal risk, analyst decision-making is represented using a simulation-based feedback loop which is periodically re-trained offline (i.e., after hours) to improve the finding up of faithful measurement of the system and minimize false positives which persist.

In summary, the prime addition of this effort can be expressed as pursue:

- Integrated, end-to-end SOC Architecture incorporating SIEM, Anomaly Detection and SOAR Alert Handling.
- Implementation of an unsupervised learning-based anomaly detection model for identifying behavioral deviations in security telemetry.
- Design and evaluation of a human-in-the-loop feedback mechanism, implemented through a simulated analyst feedback channel, to support controlled offline model refinement.
- Experimental evaluation demonstrating improved detection capability and reduced alert burden using CICIDS-style datasets and synthetic authentication activity.

The framework is intended to balance automation efficiency with operational safety, enabling practical exploration of intelligent security operations in a controlled research setting.

II. RELATED WORK

Traditional IDS have used mark positioned IDS and anomaly-based IDS as a means of intrusion detection. name-based IDS utilize predefined IDS rules and known attack signatures in order to detect previously identified attacks, however they are ineffective at identifying new and evolving attack methods. Anomaly-based IDS have utilized statistical

and machine learning techniques to determine an individual's "normal" behavior in their network environment; therefore, once that "normal" behavior is determined, the system can then determine when deviations occur.

Early anomaly detection techniques were based on statistical profiling and clustering to determine legitimate traffic versus anomalous behavior. The use of K-Means Clustering and PCA, for example, has allowed researchers to create a lower dimensional representation of their data that is able to highlight traffic anomalies. These types of methods are generally not effective without careful tuning of parameters to account for the specific characteristics of the organization's network.

More recent studies have investigated tree-based and ensemble learning methods for intrusion detection. Isolation Forest has received particular attention because it isolates anomalies by randomly partitioning data rather than learning explicit class boundaries. This property makes it suitable for high-dimensional and unlabeled security datasets, where attack labels may be incomplete or unavailable. Several works have shown that Isolation Forest can effectively detect network scanning, denial-of-service behavior, and authentication anomalies with relatively low computational overhead.

In addition to the detection capabilities of SIEM systems are used for SIEM to collect information from many types of things into a single location where they can be viewed together. Huge quantity of information can be ingested, indexed and visualized using systems that are built around the elastic stack which will help analysts find and investigate unusual activity. The use of SOAR has also developed platforms for automating repetitive incident response tasks including ticketing, alert correlation and workflow activation.

Despite these advances, most existing solutions treat detection, monitoring, and response as separate components. Anomaly detection models are frequently evaluated offline, while SIEM platforms primarily focus on visualization and alerting, and SOAR systems execute predefined automation playbooks without adaptive learning. Only limited research considers closed-loop SOC architectures in which analyst decisions influence future detection behavior. Incorporating analyst feedback is important because security environments evolve continuously, and static models quickly become outdated.

The framework developed in this paper is different than previous frameworks because it incorporates Anomaly Detection, SIEM based Monitoring, Automated Alert Handling, and Analyst Guided Refinement as an integrated Operational Pipeline. A human-in-the-loop feedback mechanism enables gradual adaptation to changing conditions that can help maintain operational safety, which addresses the existing gap between intrusion detection research in academia, and SOC operations.

III. PROPOSED FRAMEWORK

A. System Overview

By that I mean, the suggested architecture represents a more practical naive SOC monitoring workflow than a self centered machine learning experiment. This is because the system merges log collection, anomaly analysis, investigation support

and controlled response assistance into one single operational pipeline. The objective is to assist analysts in identifying suspicious behavior while maintaining human supervision over critical security decisions.

Security events are collected from network flow records and authentication logs. These events are preprocessed and grouped into temporal windows to capture behavioral patterns instead of individual packets or single log entries. The processed events are then analyzed using an anomaly detection model that identifies deviations from normal activity.

- Data ingestion and preprocessing
- Anomaly detection and alert generation
- SIEM-based visualization and investigation
- SOAR-assisted alert handling and analyst-guided review

After detection, suspicious events are indexed into the ELK Stack, enabling centralized monitoring and visual analysis through dashboards. Selected high-severity alerts are forwarded to the Shuffle SOAR module through webhook-based workflows, which execute predefined non-destructive actions such as alert notification and case tracking. No containment or enforcement action is performed automatically, and any potentially impactful response remains subject to explicit analyst approval.

To evaluate adaptive behavior safely, analyst decisions are represented through a simulated feedback mechanism and incorporated during scheduled offline retraining cycles. This feedback loop allows the system to study how validated outcomes can improve future detection behavior while preventing unsafe live model updates. The resulting architecture balances automation efficiency with human oversight, supporting a reliable and practical SOC prototype.

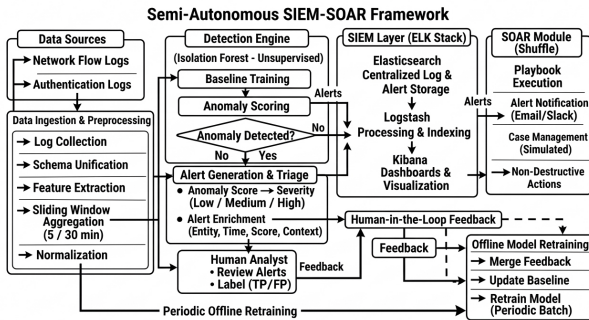


Fig. 1. Semi-autonomous SIEM-SOAR framework architecture.

The architecture represents a semi-automated operational prototype. The SOAR component performs notification and tracking actions only and does not execute containment or enforcement actions on production systems. Analyst feedback is simulated for evaluation and incorporated into periodic offline retraining rather than real-time adaptive learning.

B. Data Sources and Ingestion

The framework ingests security telemetry from two primary sources in order to represent both network activity and user behavior.

1) Network Flow Logs: Network traffic data was extracted from flow records based upon CICIDS-type flow records. Each record contains connection-level statistics, for example originating IP address and receiving IP address, service ports, and communication protocol metadata, the time period for which the flow lasted, the number of incoming or outgoing data in packets and the amount of bytes transferred in the flow. Flow-based monitoring is able to analyze an individual's communication patterns by way of examining the flow level data and does so without the need to examine the content of the packets being transmitted.

2) Authentication Logs: Authentication events consist of synthetically generated SSH-style login records, including successful and failed login attempts. These records contain fields such as timestamp, user identity, source IP, host, authentication type, and login status, enabling the analysis of suspicious account behavior such as brute-force attempts or anomalous access patterns.

All incoming logs are parsed and normalized into a unified schema. Timestamp formats, entity identifiers, and feature representations are standardized to ensure consistency across heterogeneous sources. This normalization enables correlation between network communication behavior and authentication activity within a single analytical pipeline.

```

suricata@suricata01:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
11/18/2024 11:20:11 - suricata - Running suricata under test mode
11/18/2024 11:20:11 - suricata - CPU/cpus online: 4
11/18/2024 11:20:11 - suricata - Fast output device (regular) initialized: fast.log
11/18/2024 11:20:11 - suricata - eve-log output device (regular) initialized: /var/log/suricata/eve.json
11/18/2024 11:20:11 - suricata - state output device (regular) initialized: state.log
11/18/2024 11:20:11 - suricata - 1 rule files processed: 47461 rules successfully loaded, 0 rules failed
11/18/2024 11:20:11 - suricata - Threshold config parsed: 0 rules/1 found
11/18/2024 11:20:11 - suricata - 47461 signatures processed, 1251 are IP-only rules, 1108 are inspecting packet payload, 48016 inspect application layer, 108 are decoder e
ent only
11/18/2024 11:20:11 - suricata - cleaning up signature grouping structure... complete
11/18/2024 11:20:11 - suricata - cleaning up signature grouping structure... complete
TNG testbed:com (217.160.0.187) 50484 bytes of data
4 bytes from 217.160.0.187:elastic.ssi.ulr.com (217.160.0.187): icmp_seq=1 ttl=128 time=153 ms

... testbed:com ping statistics ...
packets transmitted: 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 152.528/152.528/152.528/0.000 ms
suricata@suricata01:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
11/18/2024 11:20:11 - suricata - Running suricata under test mode
11/18/2024 11:20:11 - suricata - CPU/cpus online: 4
11/18/2024 11:20:11 - suricata - Fast output device (regular) initialized: fast.log
11/18/2024 11:20:11 - suricata - eve-log output device (regular) initialized: /var/log/suricata/eve.json
11/18/2024 11:20:11 - suricata - state output device (regular) initialized: state.log
11/18/2024 11:20:11 - suricata - 1 rule files processed: 47461 rules successfully loaded, 0 rules failed
11/18/2024 11:20:11 - suricata - Threshold config parsed: 0 rules/1 found
11/18/2024 11:20:11 - suricata - 47461 signatures processed, 1251 are IP-only rules, 1108 are inspecting packet payload, 48016 inspect application layer, 108 are decoder e
ent only
11/18/2024 11:20:11 - suricata - cleaning up signature grouping structure... complete
11/18/2024 11:20:11 - suricata - cleaning up signature grouping structure... complete
suricata@suricata01:~$

```

Fig. 2. Suricata IDS network traffic monitoring and event logging.

C. Schema Unification and Event Correlation

Security telemetry collected from different sources often follows dissimilar formats and field structures. To allow joint analysis, all incoming logs are converted into a common event representation. Each record is mapped into a unified schema consisting of three main elements:

- Timestamp indicating when the activity occurred
- Entity identifier such as an IP address, host system, or user account
- Event attributes describing the activity, including network flow statistics or authentication results

The normalization process aligns field names, time formats, and identifiers across datasets. After transformation, events

originating from network communication and login activity can be analyzed together rather than independently.

This unified representation enables correlation of related activities across telemetry sources. For example, repeated login failures associated with a host that simultaneously generates abnormal network connections can be identified as a single suspicious behavior pattern. The correlated events are then forwarded to the aggregation stage for further behavioral analysis.

D. Sliding Window Feature Aggregation

Individual log entries represent isolated events and are often insufficient for identifying malicious behavior. Instead of analyzing records independently, the system groups unified events into fixed temporal windows. Sliding windows of 5 minutes and 30 minutes are used to capture both short-duration activity bursts and longer behavioral trends.

Within each window, multiple observations related to the same entity are aggregated to form a behavioral profile. This approach allows the system to analyze how an entity behaves over time rather than reacting to a single packet or login attempt.

For each window, a set of statistical features is computed, including:

- Number of flows and packet activity counts
- Total and average transferred byte volumes
- Number of distinct destination hosts or services contacted
- Authentication attempt counts and failed-login frequency

The aggregated representation reduces noise from individual events while highlighting persistent or unusual activity patterns. The resulting feature vectors are then provided to the anomaly detection stage for behavioral analysis.

```
saugat@saugat-virtual-machine: ~/Desktop/soc-lab$ python3 Scripts/phase5/phase5_4_window_aggregation.py
[*] Aggregating 5min windows...
write 48926 rows (total: 48926)
write 238447 rows (total: 279373)
write 49392 rows (total: 348765)
write 43232 rows (total: 391997)
write 15583 rows (total: 407580)
write 23996 rows (total: 467486)
write 22246 rows (total: 786646)
write 4365 rows (total: 791011)
write 2444 rows (total: 793455)
[*] Finished 5min aggregation
[*] Total rows: 793455
[*] Saved to: data/processed/features_5min.parquet

[*] Aggregating 30min windows...
write 38460 rows (total: 38460)
write 43416 rows (total: 53876)
write 12649 rows (total: 66525)
write 9380 rows (total: 75905)
write 4541 rows (total: 80446)
write 47624 rows (total: 128070)
write 23639 rows (total: 151709)
write 2034 rows (total: 153743)
write 1168 rows (total: 154911)
[*] Finished 30min aggregation
[*] Total rows: 154911
[*] Saved to: data/processed/features_30min.parquet
```

Fig. 3. Sliding-window feature extraction from network and authentication logs.

E. Baseline-Driven Anomaly Detection

We are not only dependent on the signatures of attacks (the detections module look for unusual patterns in the behavior). We train an Isolation Forest algorithm on baseline traffic, which exhibits what we consider normal behavior for our system. By learning the statistical characteristics of normal system behavior, the model can recognize unusual patterns that may indicate malicious actions.

This strategy avoids dependence on labeled attack datasets and allows the detection of previously unseen threats. Each aggregated feature vector is assigned an anomaly score, where

higher scores indicate a greater deviation from typical behavior. A detection threshold is determined experimentally to identify an optimal operating point at which the sensitivity of the system and the false alarm rate can be balanced. Lower thresholds increase detection capability but may generate excessive alerts, while high level verge reduce FP at the amount of missed events. The selected configuration was found to provide stable performance for operational monitoring.

An alternative model based on One-Class Support Vector Machines was also evaluated during preliminary experiments. However, it showed lower stability and higher computational cost on large log volumes, and therefore the Isolation Forest approach was retained as the primary detection mechanism.

```
saugat@saugat-virtual-machine: ~/Desktop/soc-lab$ python3 Scripts/phase5/phase5_1_score_windows.py
[*] Scoring 5min windows...
[*] Total windows: 238679
[*] Anomalies detected: 10974
[*] Saved: data/processed/alerts/alerts_5min.parquet

[*] Scoring 30min windows...
[*] Total windows: 46991
[*] Anomalies detected: 2418
[*] Saved: data/processed/alerts/alerts_30min.parquet
```

Fig. 4. Isolation Forest anomaly scoring.

IV. SIEM INTEGRATION AND ALERT MANAGEMENT

A. Elasticsearch Indexing

Detected anomalies are stored as structured alert records within an Elasticsearch index. Each alert document includes:

- Timestamp of the observed activity
- Entity identifier (IP address or host)
- Computed anomaly score
- Aggregated behavioral context derived from the sliding window features

The indexing process simulates the ingestion of alerts into a Security Information and Event Management (SIEM) platform. Storing alerts in a searchable datastore enables rapid querying, filtering, and correlation of suspicious events across time and entities. Analysts can therefore investigate patterns of abnormal activity instead of isolated alerts.

```
saugat@saugat-virtual-machine: ~/Desktop/soc-lab$ curl "http://localhost:9200/soc-alerts/_search?size=1&pretty"
{
  "took": 20,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 10000,
      "relation": "gte"
    },
    "max_score": 1.0,
    "hits": [
      {
        "_index": "soc-alerts",
        "_type": "_doc",
        "_id": "y2ITipwqlvgdh_LceVT",
        "_score": 1.0,
        "_source": {
          "timestamp": "2017-07-06T07:55:00Z",
          "entity": "test-1p",
          "window": "5min",
          "anomaly_score": 0.5,
          "severity": "high",
          "model": "IsolationForest",
          "flow_count": 30,
          "total_bytes": 5000,
          "auth_events": 1,
          "failed_auths": 1,
          "unique_dst_ips": 3
        }
      }
    ]
  }
}
```

Fig. 5. Security alert stored in Elasticsearch index.

B. Visualization using Kibana

To support operational analysis, the indexed alerts are visualized using a Kibana dashboard built on top of the Elasticsearch datastore. The dashboard allows analysts to monitor system activity in near real time and to quickly inspect abnormal behavioral patterns.

The interface provides three primary investigation perspectives:

- Temporal alert trends, showing how alert frequency changes over time
- Distribution of anomaly scores to distinguish benign deviations from suspicious behavior
- Entity-focused views that enable analysts to track activity associated with specific hosts or IP addresses

These visualizations help analysts correlate related events and identify persistent or coordinated behaviors that may not be evident from individual alerts alone. By presenting contextual information in an interactive format, the dashboard reduces manual log inspection effort and improves situational awareness during investigations.

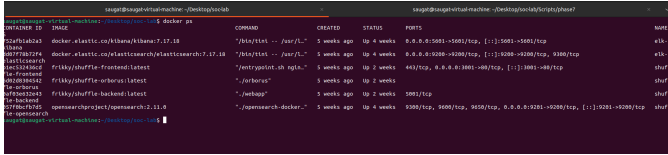


Fig. 6. Containerized Elasticsearch and Kibana SIEM deployment.

V. ALERT ENRICHMENT, TRIAGE, AND SOAR INTEGRATION

A. Alert Enrichment

Raw anomaly alerts generated by the detection module contain only behavioral deviation indicators and therefore lack sufficient context for operational decision making. To support meaningful investigation, each alert is augmented with additional descriptive metadata prior to analyst review and response handling.

First, the source and destination addresses are categorized according to network locality. IP addresses are classified as internal or external based on predefined address ranges representing the organizational network boundary. This distinction assists analysts in quickly differentiating between potential insider activity and externally originating events.

Second, an asset importance attribute is associated with each event. Because the experimental environment does not represent a production enterprise, asset criticality is modeled using a simulated priority scale. Hosts designated as servers, authentication systems, or shared infrastructure components are assigned higher importance values than general user workstations. This allows the triage process to prioritize events that may affect sensitive or operationally significant resources.

Finally, alerts are mapped to representative adversarial behavior categories inspired by the MITRE ATT&CK framework. The mapping is implemented as a simulated heuristic enrichment layer using observable characteristics of the

anomaly, such as repeated authentication failures, abnormal connection frequency, or unusual data transfer patterns. Because the experimental environment is a prototype rather than a production SOC, these tags are intended as contextual placeholders for analyst interpretation rather than authoritative threat intelligence.

The enrichment stage converts low-level anomaly indicators into structured and interpretable security alerts. By attaching simulated contextual attributes alongside anomaly scores, the system supports more effective triage and prepares alerts for SIEM review and SOAR-assisted handling.

B. Alert Triage and Analyst Validation

After contextual enrichment, alerts are evaluated and prioritized to support efficient analyst handling. Each event is assigned a severity level derived from the anomaly score together with the associated contextual attributes. Events exhibiting strong deviation from baseline behavior, particularly those affecting important assets or involving external sources, are categorized as high severity. Moderate deviations are labeled as medium severity, while minor behavioral irregularities are treated as low severity observations.

This prioritization mechanism reduces the volume of alerts requiring immediate attention and enables analysts to focus on events most likely to represent genuine security threats. Instead of manually reviewing all generated anomalies, the analyst workflow becomes risk-oriented, where high-severity alerts are investigated first and lower-severity events are reviewed opportunistically or in batches.

In the current prototype, analyst decisions are modeled through a simulated feedback channel rather than collected from live SOC operators. The feedback stage represents outcomes such as confirming suspicious behavior, dismissing benign activity, or deferring further inspection. These outcomes are later incorporated into the offline retraining workflow to improve model reliability over time.

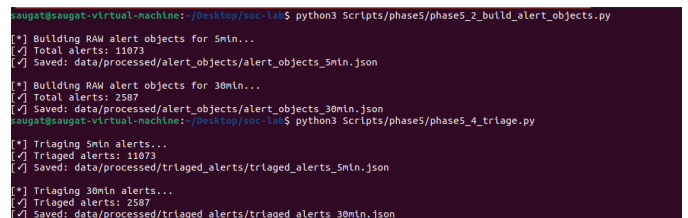


Fig. 7. Severity-based alert generation and triage.

C. SOAR Integration

In addition, as part of supporting the coordination of incident response efforts, the high severity alerts generated by this system will be transmitted to a Security Orchestration, Automation, and Response (SOAR) system. This system has been integrated with the Shuffle SOAR system via a Webhook interface and will transmit structured alert information from this system to the Shuffle SOAR system in almost real-time.

Upon receiving an alert, the SOAR platform triggers pre-defined response workflows. The implemented workflows are intentionally non-destructive and are designed to assist analysts rather than replace them. Automated actions include generating notifications, creating investigation records, and maintaining structured alert tracking for follow-up analysis.

The automation stage therefore acts as a response assistant rather than an autonomous decision maker. No destructive containment action is executed automatically. Any response that could affect operational systems remains subject to explicit analyst approval before execution.

Figure 7 shows the structured alert representation produced after enrichment and severity-based triage, which is then forwarded to the SOAR workflow for notification and case tracking.

VI. HUMAN-IN-THE-LOOP FEEDBACK AND RETRAINING

Because the prototype was evaluated in a controlled laboratory setting, analyst feedback is modeled through a simulated feedback channel rather than collected from live SOC analysts. The feedback mechanism represents analyst verdicts such as confirming a true positive or marking an alert as a false positive. These labels are stored separately from the real-time detection pipeline, and the operational model is not updated during live monitoring.

Instead, the simulated feedback is incorporated during scheduled offline retraining cycles. During retraining, validated observations are used to update the baseline representation, after which a new Isolation Forest model is trained and evaluated before reuse.

This human-in-the-loop design provides three practical benefits:

- Reduction of persistent false positives
- Adaptation to evolving behavioral patterns
- Incremental improvement of detection reliability without unsafe live model updates

The feedback loop therefore supports a semi-autonomous SOC design in which automation assists investigation, while model adaptation remains conservative, offline, and human-guided.

VII. EVALUATION METHODOLOGY

A. Dataset and Experimental Setup

The framework is evaluated using network traffic derived from the CICIDS2017 family of flow datasets together with synthetically generated authentication activity representing SSH login behavior. The network flows provide communication statistics such as connection frequency and transfer characteristics, while the authentication logs emulate user access attempts including both successful and failed logins. Combining these sources allows the system to model both network-level and user-level behavior within a unified analytical pipeline.

To reduce the risk of data leakage and to approximate operational deployment, a temporal train-test split is adopted. Earlier observations are used to construct the baseline behavior

model, and later observations are reserved for evaluation. This arrangement reflects a practical monitoring scenario in which a deployed system must analyze future activity that was not present during training.

B. Evaluation Metrics

The effectiveness of the detection model is evaluated by calculating the classification measures from the confusion matrix. TP is the correct detection of attacks, FP is the normal event incorrectly classified as an attack, TN is the correct classification of normal events, and FN is the attacks that were not detected.

Exactness is the amount of correctly classified attacks out of the total attacks generated:

C. Results

Table I presents a comparison of detection performance before and after the feedback-driven retraining stage.

TABLE I
DETECTION PERFORMANCE COMPARISON

Model	Precision	Recall	F1-score	ROC-AUC	FPR
IF (Baseline)	0.29	0.45	0.36	0.996	0.002
IF (Retrained)	0.26	0.89	0.41	0.996	0.005

```

python Scripts/phase2/phase2_evaluate_model.py
[*] Loading test data...
[*] Feature columns used:
['flow_count', 'total_bytes', 'total_packets', 'mean_duration', 'unique_dst_ips', 'auth_events', 'failed_auths', 'successful_auths', 'entity_id']

===== Original Isolation Forest =====
Precision : 0.290
Recall : 0.450
F1-score : 0.359
ROC-AUC : 0.996
FPR : 0.002

===== Retrained Isolation Forest =====
Precision : 0.262
Recall : 0.893
F1-score : 0.500
ROC-AUC : 0.996
FPR : 0.005

```

Fig. 8. Performance metrics of baseline and retrained Isolation Forest models.

The retrained model shows a substantial increase in recall, indicating an improved ability to identify malicious activity. Although precision decreases slightly, the false positive rate remains low and manageable for operational monitoring. In practice, missing an attack is typically more critical than reviewing additional alerts; therefore, the observed trade-off is acceptable. These results demonstrate that incorporating simulated analyst feedback can enhance detection sensitivity while maintaining practical usability in a monitoring-oriented prototype.

VIII. ETHICAL CONSIDERATIONS AND SAFETY

The framework is designed with operational safety as a primary requirement. Automated response mechanisms are restricted to non-destructive actions, and no containment or remediation step is executed without human verification. Instead of directly blocking hosts or disabling accounts, the system limits automation to notification, case tracking, and response-assistance workflows, while any impactful action is assumed to require analyst approval.

Escalation procedures therefore remain under analyst control, ensuring that legitimate services are not disrupted by incorrect automated decisions. In addition, the enrichment process uses simulated metadata and avoids processing personally

identifiable information. The design emphasizes transparency by providing interpretable alert context and anomaly scores, allowing analysts to understand why an event is considered suspicious. As a result, the system supports explainable and responsible security automation.

IX. LIMITATIONS AND FUTURE WORK

Although the framework demonstrates how to operate an actual SOC, there is still some work to be done in terms of improving SOC operation. There are still areas that are partially simulated (enrichment layer, authentication telemetry, and analyst feedback channel) and thus are not representative of the live Enterprise operations. In addition, all testing was completed using CICIDS-style traffic which is not going to represent all possible environments. Thus, the current system should be considered a prototype for research and development purposes rather than a deployed enterprise defense platform.

Future work will focus on integrating external intelligence sources such as IP reputation services and vulnerability feeds. Additional improvements include exploring multi-model ensemble detection to combine statistical and behavioral learning approaches. Finally, deployment in a live operational environment will enable measurement of system latency, analyst interaction patterns, and long-term adaptation performance under real traffic conditions.

X. CONCLUSION

This paper presented a semi-automated SOC framework combining an anomaly detection layer with centralized alerting, controlled remediation assistance, and analyst guided feedback. The system processes CICIDS-style network flow telemetry together with synthetically generated authentication activity, aggregates behavior using temporal sliding windows, and applies an unsupervised Isolation Forest model to identify deviations from baseline behavior.

Experimental results show that by using simulated analyst feedback to train the model during offline intervals of time, detection sensitivity increases greatly. As such, the model is able to detect anomalies with much greater accuracy than it was before the training and maintain an acceptable false positive rate; as such, the system generates fewer alerts regarding anomalous activity (and thus less "noise") but identifies anomalous behavior more effectively.

The architecture does not rely on labeled attack datasets and avoids destructive automated actions. Instead, it combines automated analysis with supervised operational oversight, enabling safe deployment in monitoring-oriented environments. By centralizing alerts in a SIEM platform and supporting structured investigation workflows, the framework assists analysts while preserving human decision authority.

Therefore, this research presents evidence that human-in-the-loop systems are effective at bridging the gap from purely manual monitoring to automated security systems using machine learning and provide an actionable methodology for using machine learning in operational security monitoring systems while ensuring reliability and safety of operation.

REFERENCES

- [1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [2] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [3] Elastic, "Elastic Stack Overview," 2023. [Online]. Available: <https://www.elastic.co/elastic-stack>
- [4] MITRE Corporation, "MITRE ATT&CK Framework," 2023. [Online]. Available: <https://attack.mitre.org/>
- [5] J. Brown and S. Gommers, "Security Orchestration, Automation, and Response (SOAR): Current Trends," *IEEE Security & Privacy*, vol. 18, no. 5, pp. 72–77, 2020.
- [6] M. Conti, T. Dargahi, and A. Dehghantanha, "Human-in-the-Loop Security Systems: A Survey," *ACM Computing Surveys*, vol. 54, no. 1, 2021.
- [7] D. E. Denning, "An Intrusion-Detection Model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.
- [8] S. Axelsson, "The Base-Rate Fallacy and Its Implications for the Difficulty of Intrusion Detection," in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 1999, pp. 1–7.
- [9] W. Lee and S. J. Stolfo, "Data Mining Approaches for Intrusion Detection," in *Proc. USENIX Security Symposium*, 1998, pp. 79–93.
- [10] M. Roesch, "Snort: Lightweight Intrusion Detection for Networks," in *Proc. USENIX LISA*, 1999, pp. 229–238.
- [11] Open Information Security Foundation, "Suricata: Open Source IDS/IPS/NSM Engine," 2023. [Online]. Available: <https://suricata.io>
- [12] N. Moustafa and J. Slay, "The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Dataset," in *Proc. IEEE Military Communications Conference*, 2015, pp. 1–6.
- [13] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [14] A. Patcha and J.-M. Park, "An Overview of Anomaly Detection Techniques: Existing Solutions and Latest Technological Trends," *Computer Networks*, vol. 51, no. 12, pp. 3448–3470, 2007.
- [15] C. C. Aggarwal, *Outlier Analysis*, 2nd ed. Springer, 2017.
- [16] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [17] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.
- [18] R. Behl and K. Behl, *Cybersecurity and Cyberwar: What Everyone Needs to Know*. Oxford University Press, 2017.
- [19] A. Oprea, Z. Li, and K. Bowers, "Intrusion Detection and Alert Correlation in Security Operations Centers," *IEEE Security & Privacy*, vol. 16, no. 4, pp. 46–55, 2018.
- [20] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, "An Empirical Comparison of Botnet Detection Methods," *Computers & Security*, vol. 45, pp. 100–123, 2014.